

Customer360: Turning Data into Insights for Digital Excellence using Recommendation System

Digvijay Kewale, Ravi Kumar Gupta, Abhishek Satpathy, Pritpal Singh Sehgal, Sagar Mayya J N*

Department of Data Science, Machine Learning & Artificial Intelligence, PES University, Bengaluru, Karnataka - 560100, India

Abstract

Recommendation systems play a crucial role in enabling personalized user experiences across a wide range of digital platforms, including e-commerce websites, online streaming services, and digital content libraries. These systems help users discover relevant items—such as products, movies, or articles—by intelligently predicting their preferences based on historical interactions, content features, or similarities with other users.

In this work, we introduce Customer360, a flexible and extensible hybrid recommendation framework. The primary objective of Customer360 is to provide domain-agnostic, scalable recommendations while maintaining high accuracy and responsiveness.

As part of the current implementation, several models and techniques were explored and evaluated. Specifically, we experimented with K-Means clustering for user segmentation and Singular Value Decomposition (SVD) for collaborative filtering. Among the various configurations tested, the combination of K-Means clustering followed by SVD-based matrix factorization demonstrated the most promising performance under the existing computational infrastructure. This hybrid approach effectively balances user groupings with item preference modeling, resulting in meaningful and context-aware recommendations.

To enhance scalability and reduce response time in real-world deployment scenarios, we propose the integration of Approximate Nearest Neighbor (ANN) search using FAISS (Facebook AI Similarity Search) in future iterations. FAISS is particularly suitable for handling large-scale vector similarity searches and will enable efficient retrieval of top recommendations with low latency.

Further planned enhancements include incorporating advanced transformer-based language models such as BERT, which can offer deeper semantic understanding of user queries and item metadata. Additionally, we aim to integrate user feedback mechanisms, allowing the system to adapt and improve recommendations over time based on user interactions. Multilingual support is also a key feature on our roadmap, which will broaden the system's applicability across different regions and user bases, enhancing accessibility and personalization at a global scale.

Received on August 2025; accepted on XXXX; published on XXXX

Keywords: Recommendation System, TF-IDF, FAISS, Embeddings, BERT, Recall@K, Precision@K, RMSE, Content-based Filtering.

Copyright © 2025 Digvijay Kewale *et al.*, licensed to ICST. This is an open access article distributed under the terms of the Creative Commons Attribution license (<http://creativecommons.org/licenses/by/3.0/>), which permits unlimited use, distribution and reproduction in any medium so long as the original work is properly cited.

doi:10.4108/XX.XX.XX

1. Introduction

Across sectors — from e-commerce to entertainment — the recent trend implies, if not demands, showcasing

some form of recommendations as a part of the digital value chain. It helps users to find products or services that meet their preferences, behavioral past and similarity with other users or items.

While collaborative filtering and content-based filtering are some of the top recommendations, [1] we

*Corresponding author. Email: digvijay.kewale@gmail.com

have seen it not ideal for scenarios that include cold-restart problem and scalability. This information can be very difficult to collect for collaborative filtering in particular. Content-based filtering, on the other hand, is very reliant on item metadata and consequently can suffer from a severe lack of personalization.

Although collaborative filtering and content-based filtering are some of the top recommendations, we have seen it not ideal for scenarios that includes cold-restart problem and scalability. This information can be very difficult to collect for collaborative filtering in particular. Content-based filtering, on the other hand, is very reliant on item metadata and consequently can suffer from a severe lack of personalization.

The proposed system, named **Customer360**, integrates multiple recommendation strategies to improve personalization.

The implemented components include: (i) a K-Nearest Neighbors (KNN) model for collaborative filtering, (ii) a hybrid K-Means clustering with Singular Value Decomposition (SVD) for user and item segmentation, (iii) content-based filtering using TF-IDF to identify relevant keywords in item descriptions, and (iv) a Neural Collaborative Filtering (NCF) model to capture nonlinear user-item interaction patterns.[2]

Planned extensions involve incorporating BERT (Bidirectional Encoder Representations from Transformers) to model deeper semantic relationships and FAISS (Facebook AI Similarity Search) for efficient retrieval in large vector spaces. This architecture is particularly suited for cold-start scenarios with limited historical interaction data.

The system is evaluated using standard recommendation metrics such as Root Mean Square Error (RMSE), Recall@10, and Precision@10, demonstrating its effectiveness in personalized item retrieval.

2. Related works

2.1. Understanding techniques and approaches in Recommendation systems

Building effective recommendation systems is considered a challenging task, as it involves combining different techniques and carefully addressing the specific needs of the system and the data being used.

Basic deep learning methods and important data components are seen as essential parts of this process. Therefore, it is important that these elements—and how they interact—are well understood in order to clearly grasp how recommendation systems are developed. In modern deep learning-based systems, a mix of various methods is often used to improve both accuracy and personalization. Each method plays a unique role in helping the system perform better.

In the following sub-subsections, the main techniques and approaches are introduced in a step-by-step way, starting with the basics and moving toward more advanced methods. This structure helps to show how different techniques can be supported or based on each other to improve the overall system.

2.2. Collaborative filtering

Collaborative filtering is a common strategy used in recommendation systems, and matrix decomposition is one method used to carry it out. This approach focuses on making personalized suggestions by looking at user preferences and finding patterns among users with similar interests or behaviors. By grouping users with similar traits, the system can provide recommendations based on the choices and feedback of others in the same group. This method works well because people often prefer to consider the opinions of others before making their own decisions. Collaborative filtering technology must include the following aspects:

- (1) A process of comparing and collecting the interest preferences of each user;
- (2) A lot of user information to predict personal interest preferences;
- (3) Statistics on the degree of relevance of interest preferences to develop recommendations for users who have the same interest preferences.

Recommendation systems are generally divided into two main types: content-based and collaborative filtering systems. In addition, some recommendation methods are designed to adapt to specific contexts or make use of information such as users' past behavior over time or their demographic data. Content-based systems focus on suggesting items that are similar to those a user has previously searched for, viewed, purchased, or rated positively. To do this effectively, the system must be able to identify and compare similarities between different items. [3]

2.3. Content-based filtering

Content-based filtering is a recommendation technique that suggests items to users based on the attributes of items they have previously interacted with. Each item is represented by a set of features—such as category, description, tags, or textual metadata—and each user is associated with a user profile constructed from the features of items they have rated or viewed in the past.

The core idea is that if a user has shown interest in certain items, the system should recommend other items with similar characteristics. This similarity is typically computed using methods like cosine similarity, Euclidean distance, or machine learning techniques that operate on item feature vectors.

A key advantage of content-based filtering is its ability to provide personalized recommendations without

relying on other users' data. This makes it particularly effective in scenarios with sparse interaction data or for new users (cold-start users). However, its reliance on item features can lead to over-specialization, where the system only recommends items that are very similar to past preferences, limiting diversity.

2.4. Hybrid models

Hybrid recommendation systems combine two or more recommendation techniques to leverage their individual strengths and address their respective limitations. Depending on the characteristics of the data and the application domain, various hybridization strategies are employed—most commonly integrating content-based (CB) and collaborative filtering (CF) methods.

Collaborative filtering relies on identifying similarities in user behavior, such as access patterns and navigation paths among learners. In contrast, content-based filtering focuses on analyzing the relationship between item features and a user's historical preferences. By merging these approaches, hybrid systems can generate more accurate and diverse recommendations.

This fusion not only mitigates the cold-start problem and sparsity issues but also expands the recommendation scope by enabling cross-domain predictions. As a result, hybrid systems are particularly effective in complex environments like e-learning, where user intent, content relevance, and interaction history must all be considered to deliver personalized and meaningful suggestions.

2.5. Advantages and Disadvantages of each models

2.6. Deep Learning-based Approaches

In recent years, recommendation systems have played an important role in helping businesses enhance customer satisfaction and increase revenue, especially in fields such as e-commerce and entertainment.[4]

Deep learning methods, including graph neural networks, convolutional neural networks, and recurrent neural networks, have been widely used in recommendation systems.[5] In contrast to linear models, deep neural networks are able to capture nonlinear patterns in data by utilizing nonlinear activation functions like ReLU, sigmoid, and tanh. This ability enables them to represent complex and subtle relationships between users and items more effectively.[6]

Recommendation systems have demonstrated strong performance by applying deep learning methods in different fields such as social networks, e-commerce, and e-learning.

Various architectures are used in deep learning-based recommendation systems. For example, autoencoder-based systems are valued for their ability to reduce

Table 1. Advantages and Disadvantages of Recommendation Techniques

Method	Advantages	Disadvantages
Collaborative Filtering	- Learns user preferences from interactions. - Can suggest novel items based on user similarity. - Does not require item metadata.	- Cold-start problem for new users or items. - Struggles with sparse user-item matrices. - Biased toward popular items.
Content-Based Filtering	- Requires minimal user data to get started. - Handles new items well using metadata. - Generates personalized recommendations.	- May lead to over-specialization. - Limited novelty in suggestions. - Depends on quality of item features.
Hybrid Models	- Combines strengths of multiple methods. - Mitigates cold-start and sparsity issues. - Provides better diversity and accuracy.	- Increased system complexity. - Higher computational cost. - Requires multiple data sources.

data dimensionality, extract important features, and reconstruct input data. Embedding techniques transform raw data into vector representations, which are then processed by neural networks. These networks may include feedforward, convolutional, or recurrent types, each designed to address different aspects of recommendation tasks. Collaborative filtering neural networks generate recommendations by identifying similarities between users or items. Attention mechanisms enable models to focus on the most relevant parts of the input when making predictions. Lastly, reinforcement learning optimizes recommendations by learning from user interactions and feedback.[7][8][9]

2.7. Greedy Algorithms

Global Greedy (G-Greedy): The Global Greedy algorithm starts with no suggestions. Then, it looks at every single possible suggestion it could make to anyone, for anything, at any point in time. It chooses the one suggestion that it believes will bring in the most money immediately. It makes that suggestion. Then, it repeats

the process: it looks at all the remaining possible suggestions and picks the one that will make the most money right away, and it keeps doing this. This continues until it has made enough suggestions (maybe there's a limit) or the time runs out. It's all about picking the suggestion that seems most profitable at each individual step, without necessarily thinking about how those suggestions fit together over time.[10]

Sequential Local Greedy (SL-Greedy): The Sequential Local Greedy (SL-Greedy) algorithm initiates without any pre-existing recommendation strategies. It operates with a focus on temporal progression, proceeding through the recommendation timeline in a defined chronological sequence. At each discrete time point, SL-Greedy assesses all viable recommendation options for all relevant users and items at that specific moment. The algorithm evaluates the immediate revenue potential of each possible recommendation within the current time step, while also considering the recommendations already implemented in prior time steps. Subsequently, it selects the recommendation that yields the highest immediate increase in revenue for the present time frame, incorporates this selection into its strategy set, and then moves to the next time step in the sequence. This localized, step-by-step decision-making process ensures that the algorithm adheres to the temporal flow of recommendations, enabling it to adapt its choices based on the history of previously made recommendations.

Randomized Local Greedy (RL-Greedy): The Randomized Local Greedy (RL-Greedy) algorithm, similar to SL-Greedy, processes recommendations in a time-ordered manner across the defined horizon. However, it differs by incorporating a degree of randomness in its selection process. At each time step, RL-Greedy identifies a set of promising recommendation options for the pertinent users and items. Subsequently, it assigns a probability weight to each of these options, directly related to their expected revenue contribution, thus favoring recommendations with higher potential gains. The algorithm then probabilistically selects one recommendation from this weighted distribution, integrates it into the evolving strategy, and proceeds to the subsequent time step. This introduction of stochasticity enables RL-Greedy to explore a broader range of potential solutions beyond the strictly optimal immediate choice, thereby reducing the susceptibility to suboptimal local solutions that a purely deterministic approach might encounter.

2.8. NLP-Based Recommendation System

The figure(1) shows a deep learning model that generates personalized reviews to improve recommendations. It uses user, item, and rating information as input, which

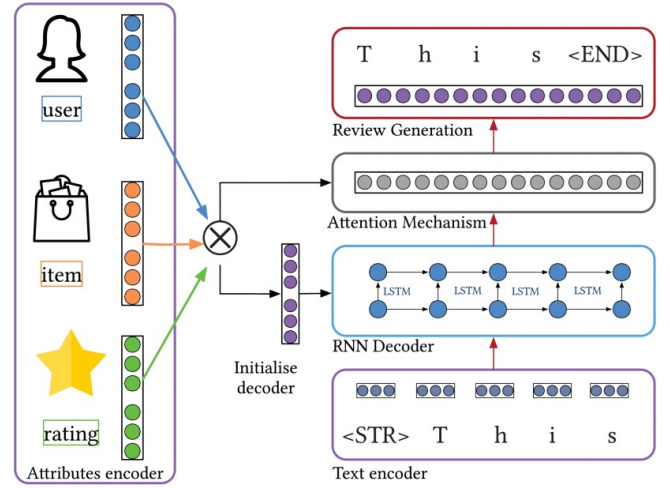


Figure 1. NLP-based architecture

are encoded into vectors. These vectors are combined to help the model understand the user's preferences.

A text encoder reads the input text and sends it to an RNN decoder, which is supported by an attention mechanism. This helps the model focus on important parts of the text while generating reviews. The output is a sentence, such as a review, which explains why an item is recommended.

The system functions as follows:

- **Attribute Encoding:** User, item, and rating features are each encoded into vector representations using embedding layers.
- **Initialization:** These embeddings are combined (e.g., concatenated or via element-wise operations) to initialize the hidden state of the decoder.
- **Text Encoder:** The input textual sequence (e.g., "This") is encoded via an LSTM-based encoder to capture its contextual meaning and convert it into hidden state vectors.
- **RNN Decoder with Attention:** An LSTM-based decoder uses the initialized hidden state to begin the review generation process. Attention mechanism is used to focus on relevant parts of the encoded input sequence during decoding.
- **Review Generation:** Finally, the model generates output text token-by-token producing a synthetic review that aligns with the user's preferences and the item attributes.
- **RMSE (Root Mean Square Error):** To assess the accuracy of predicted ratings.

The use of techniques like TF-IDF for text vectorization, combined with LSTM, RNN, attention mechanism, and encoder-decoder frameworks, provides a powerful

and explainable recommendation engine. The review generation adds a **human-interpretable layer** to the otherwise opaque recommendation process, which can significantly improve user trust and satisfaction.

3. Methodology

Table 2. Research Methodology Summary

Phase	Description
Problem Identification	Define the goal of building a recommendation system to deliver real-time and personalized product suggestions using behavioral and content data.
Data Acquisition	Collect user logs, item metadata, user profiles, and external datasets to form a comprehensive data source.
Data Preparation	Clean and encode the data, extract features using TF-IDF or embeddings, and split into training and test sets.
Model Building	Develop multiple models: • Collaborative Filtering (e.g., User-KNN, SVD) • Content-Based Filtering (TF-IDF + Cosine) • Deep Learning (e.g., NCF, BERT + FAISS) • Hybrid Models (e.g., NCF + BERT Fusion)
Evaluation	Use metrics such as RMSE, Precision@K, and Recall@K[11] to evaluate the model’s performance.
Deployment	Deploy the selected model via a REST API (FastAPI/Flask) and create a front-end UI using React for recommendation display.
Feedback Loop	Capture user clicks and interactions to continuously retrain and improve the model over time.

3.1. Dataset

To support model training and evaluation, we created a tailored dataset derived from the publicly available *Retail_Transaction_Dataset* on Kaggle.[12] The processed dataset is divided into three components:

- **Product Dataset:** Consists of distinct product names, serving as the item catalog used across all transaction records.
- **User Dataset:** Contains anonymized customer profiles with attributes such as Customer_Name, Customer_Type, Age, and Gender. Age values are assigned within logical ranges based on customer type (e.g., students: 18–25). Gender is inferred using a language model [13] based on first names, and a globally unique identifier replaces actual names for privacy.
- **Interaction Dataset:** Represents user-product interactions as binary ratings (1 if a user purchased a product, 0 otherwise). This dataset forms the core input for the recommendation models by simulating realistic customer purchase behavior.

3.2. System Architecture

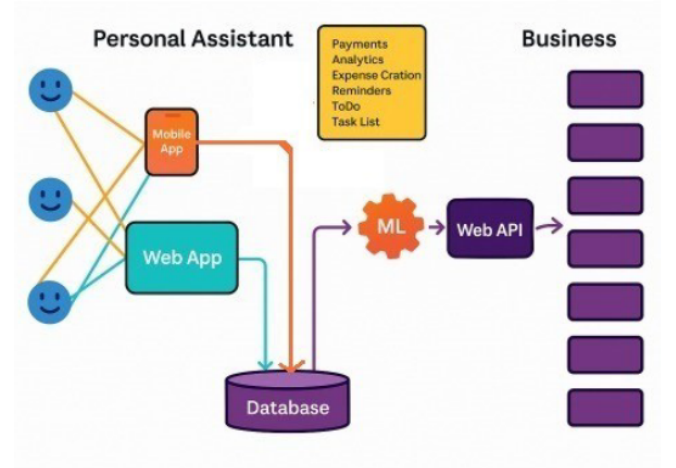


Figure 2. Overview of the architecture

- **SVD:** SVD in recommendation systems is a way to break down the complex user-item rating matrix into simpler, interpretable parts that capture hidden patterns, allowing us to predict unknown ratings and make personalized recommendations.[14][15] Given a user-item interaction matrix $R \in \mathbb{R}^{m \times n}$, SVD factorizes it as:

$$R \approx U \Sigma V^T$$

Each user u_i and item v_j are embedded in a latent space of dimension k , and predicted ratings are obtained by:

$$\hat{R}_{ij} = U_i^T V_j$$

- **TF-IDF:** Generates sparse vector representations of textual product descriptions.[16].

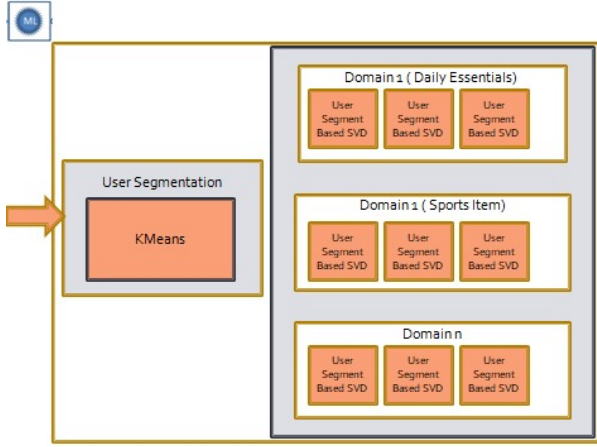


Figure 3. Detail ML Architecture Overview of Customer360

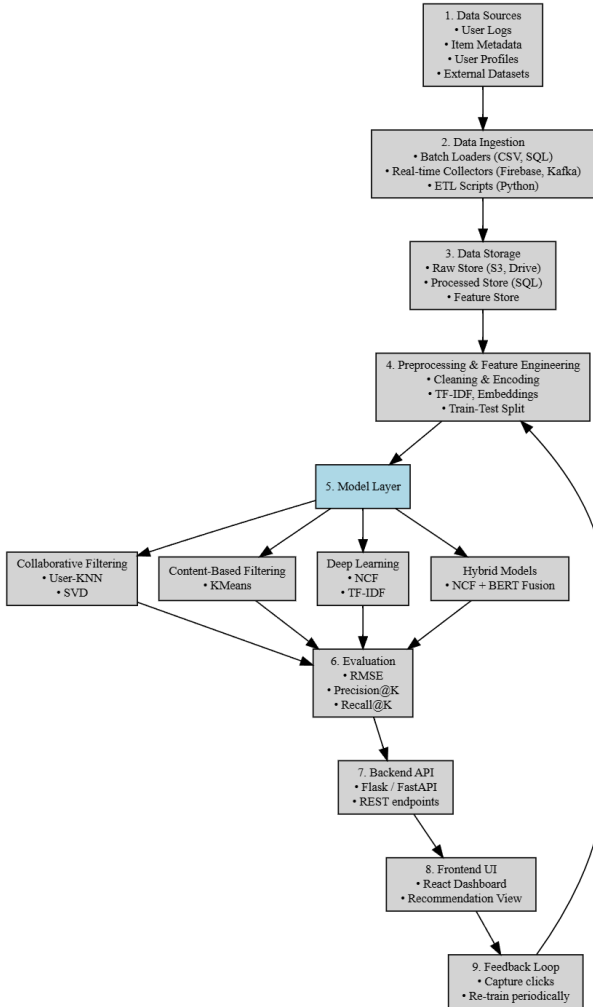


Figure 4. System architecture of the proposed recommendation system.

$$(i, u, U) = (i, u) \times (i, U)$$

$$(i, u) = \frac{f_{i,u}}{\sum_{i' \in I_u} f_{i',u}}$$

$$(i, U) = \log \left(\frac{|U|}{1 + |\{u' \in U : i \in I_{u'}\}|} \right)$$

$$(i, u, U) = \underbrace{\frac{f_{i,u}}{\sum_{i' \in I_u} f_{i',u}}}_{\text{Term Frequency (TF)}} \times \underbrace{\log \left(\frac{|U|}{1 + |\{u' \in U : i \in I_{u'}\}|} \right)}_{\text{Inverse Document Frequency (IDF)}}$$

Where:

- i = item (product, service, or content in the catalog)
- u = user
- U = set of all users
- $f_{i,u}$ = number of interactions of user u with item i (e.g., clicks, purchases)
- $|U|$ = total number of users
- $|\{u \in U : i \in I_u\}|$ = number of users who interacted with item i

- **TF (Term Frequency)** measures how often a user interacts with an item compared to their total interactions.
- **IDF (Inverse Document Frequency)** down-weights items that are popular among many users, giving more weight to niche items.
- The combined **TF-IDF score** boosts items that are personally relevant but not overly popular, improving personalization.

- **BERT Embeddings:** Captures contextual semantic features.[17]

$$x = (x) \in \mathbb{R}^d$$

where x is the input text (item description, review, or query), and x is its d -dimensional embedding vector.

The recommendation relevance score between a user u and an item i is computed using cosine similarity:

$$(u, i) = \frac{u \cdot i}{\|u\| \|i\|}$$

where u is the aggregated embedding of items the user interacted with and i is the item embedding from BERT.

- **FAISS:** Performs efficient similarity search using dense embeddings.[18]

- **Hybrid Ranking:** Combines TF-IDF and FAISS results with configurable weights.
- **NCF:** Neural Collaborative Filtering (NCF) is a deep learning-based recommendation algorithm that extends traditional collaborative filtering by replacing dot-product similarity (like in Matrix Factorization) with a learnable neural network.[19]

Let: $u = \text{UserEmbedding}(u) \in \mathbb{R}^k$ and $i = \text{ItemEmbedding}(i) \in \mathbb{R}^k$, where k is the latent dimension.

The predicted interaction score is:

$$\hat{y}_{ui} = f_{\theta}(u, i)$$

where f_{θ} is a multi-layer perceptron (MLP) that models complex user-item interactions.

The model is trained by minimizing the binary cross-entropy loss:

$$\mathcal{L} = - \sum_{(u,i) \in \mathcal{D}} [y_{ui} \log \hat{y}_{ui} + (1 - y_{ui}) \log(1 - \hat{y}_{ui})]$$

where y_{ui} is 1 if user u interacted with item i , else 0.

- **K-Means Clustering:**

[20][21]Organizes data into K groups by minimizing within-cluster variance. The procedure starts with initial centroids, assigns each data point to the closest centroid, and then recalculates each centroid as the mean of its assigned points. These steps are repeated until the cluster assignments no longer change significantly or a maximum number of iterations is reached.

Given a set of n data points $\{1, 2, \dots, n\}$, K-Means partitions them into K clusters $\{C_1, C_2, \dots, C_K\}$ by minimizing the within-cluster sum of squares (WCSS):

$$\arg \min_c \sum_{j=1}^K \sum_{i \in C_j} \|x_i - \mu_j\|^2$$

where μ_j is the centroid of cluster C_j , computed as:

$$\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$$

3.3. Model Building Approach

To enhance the effectiveness of the recommendation model, users are first clustered into different groups using K-Means. We used Elbow Plots [22] and Silhouette scores [23] to identify the clusters.

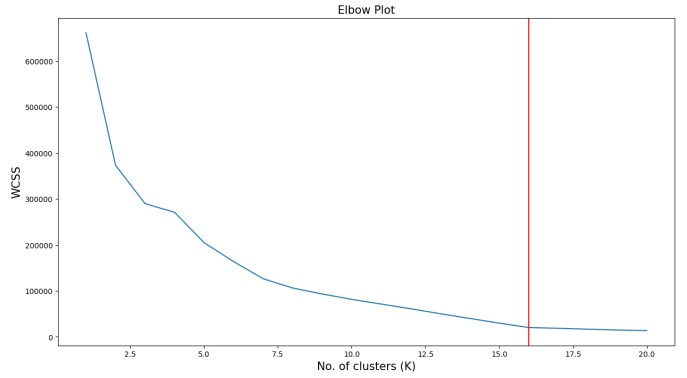


Figure 5. Finalizing Clusters using Elbow Plot

```
For 2 clusters the silhouette score is 0.3739503888522184)
For 3 clusters the silhouette score is 0.2924173902124102)
For 4 clusters the silhouette score is 0.3223853429716163)
For 5 clusters the silhouette score is 0.356077024509398)
For 6 clusters the silhouette score is 0.36112546486234337)
For 7 clusters the silhouette score is 0.496033406853414)
For 8 clusters the silhouette score is 0.5496058669159908)
For 9 clusters the silhouette score is 0.5672111246066208)
For 10 clusters the silhouette score is 0.5640348014871651)
For 11 clusters the silhouette score is 0.60187741622667)
For 12 clusters the silhouette score is 0.6269744661776101)
For 13 clusters the silhouette score is 0.6451091004794952)
For 14 clusters the silhouette score is 0.6761212709551839)
For 15 clusters the silhouette score is 0.6990296547715451)
For 16 clusters the silhouette score is 0.7339222431585742)
For 17 clusters the silhouette score is 0.7314766512688236)
For 18 clusters the silhouette score is 0.7281449934561628)
For 19 clusters the silhouette score is 0.7216494664419149)
For 20 clusters the silhouette score is 0.7165496394606081)
```

Figure 6. Silhouette Scores for identifying clusters

When a query is received from a business client, the system first identifies the appropriate user group and domain associated with the request. It then routes the query to the corresponding recommendation model tailored for that specific group and domain.

3.4. Evaluation Metrics

- **Root Mean Square Error (RMSE)**[24]: Measures error in predicted relevance scores.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

- **Recall@K:** Measures how many relevant items appear in the top K recommendations.

$$\text{Recall@K} = \frac{|\text{Recommended}_K \cap \text{Relevant}|}{|\text{Relevant}|}$$

- **Precision@k:** Recall, on the other hand, is used to evaluate the extent to which the model's positive predictions correctly identify actual positive instances.[25]

$$\text{Precision@K} = \frac{|\text{Recommended}_K \cap \text{Relevant}|}{K}$$

4. Experiments and Results

We started our approach by applying the K-Means algorithm to segment the data set into distinct user groups. After performing multiple iterations and evaluating the cluster performance using the silhouette score, we observed that the optimal configuration was achieved when the data were partitioned into 16 clusters. This configuration yielded the highest silhouette score among all the values tested, indicating well-separated and cohesive groupings. Consequently, we adopted the 16-cluster solution for further analysis and recommendation model development. Below are the silhouette scores for different numbers of clusters.

Table 3. Model Comparison using silhouette score for different clusters

No. of Clusters	Silhouette Score
2 Clusters	0.3739503888522184
3 Clusters	0.2924173902124102
4 Clusters	0.3223853429716163
5 Clusters	0.356077024509398
6 Clusters	0.36112546486234337
7 Clusters	0.496033406853414
8 Clusters	0.5496058669159908
9 Clusters	0.5672111246066208
10 Clusters	0.5640348014871651
11 Clusters	0.60187741622667
12 Clusters	0.6269744661776101
13 Clusters	0.6451091004794952
14 Clusters	0.6761212709551839
15 Clusters	0.6990296547715451
16 Clusters	0.7339222431585742
17 Clusters	0.7314766512688236
18 Clusters	0.7281449934561628
19 Clusters	0.7216494664419149
20 Clusters	0.7165496394606081

Following the clustering stage, we proceeded to experiment with various recommendation algorithms to train our recommender system. Our initial attempt involved using the nearest neighbors (KNN) algorithm. However, the performance metrics indicated suboptimal results, suggesting that KNN may not be well-suited for our dataset or task objectives.

Subsequently, we explored the Singular Value Decomposition (SVD) method, a matrix factorization technique commonly used in collaborative filtering. This approach produced significantly better results, demonstrating its effectiveness in capturing latent relationships within the user-item interaction data.

In addition to traditional models, we also experimented with a deep learning-based method known as Neural Collaborative Filtering (NCF). This model integrates neural network architectures to learn user-item interactions in a non-linear fashion. While NCF showed potential during training, post-training evaluation revealed signs of overfitting, suggesting that the model may have captured noise or failed to generalize well to unseen data.

A comprehensive comparison of all the models, including their performance across various evaluation metrics, is presented in the table below.

Table 4. Model Comparison using RMSE, Precision@10, and Recall@10

Model	RMSE	Precision@10	Recall@10
User-Based KNN	1.12	0.12	0.59
Matrix Factorization (SVD)	0.26	0.21	0.08
Content-Based (TF-IDF)	0.77	0.40	1.00
Neural CF (NCF)	0.00	0.02	0.12

5. Discussion and Conclusion

5.1. Discussion

Interpretation of Key Findings. The evaluation of various recommender system models across clustered user groups revealed that Singular Value Decomposition (SVD) performed better than KNN in our dataset. Although Neural Collaborative Filtering (NCF) initially showed promise, it exhibited overfitting during testing, indicating the need for more regularization or more data. These results suggest that traditional matrix factorization techniques remain reliable for moderately sized, sparse datasets.

Relation to Existing Work. The observed performance of SVD aligns with previous studies, which have consistently shown it to be effective for collaborative filtering tasks, especially when explicit feedback (e.g., purchase history) is available. Unlike earlier domain-specific recommender systems, our approach proposes a more adaptable framework that can generalize across domains, extending the capabilities of recommender systems beyond isolated use cases.

Practical Implications. By structuring the system as a modular web API, businesses can integrate personalized recommendations into their platforms without managing the underlying model training and infrastructure. This “recommender system as a service” (RSaaS) model is particularly useful for startups or SMEs seeking quick deployment and scalability without extensive in-house development.

Limitations. While the proof of concept demonstrates feasibility, the system’s performance may vary significantly across different domains and user behaviors. Additionally, the current cold start mitigation strategy is limited to basic clustering, which may not capture complex user preferences.

5.2. Conclusion

This work introduces a generic domain-independent recommender system built primarily on collaborative filtering and user clustering techniques. The system is designed to adapt across different business domains by selecting the most suitable recommendation model. One of the core strengths of the proposed solution is its ability to reduce the cold start problem through initial user grouping and general behavior modeling. It is envisioned as a Recommendation System as a Service (RSaaS), enabling businesses - especially startups or those with limited technical resources - to integrate recommendation capabilities without building or maintaining the underlying infrastructure. A functional proof-of-concept (PoC) has been developed, and part of the system is exposed via a web API. Future improvements include collecting richer user data, such as spending habits, interests, and lifestyle, possibly through a dedicated mobile application. This would enhance user clustering and allow for more personalized recommendations. Additionally, domain-specific tuning can involve choosing between collaborative, hybrid, or neural models such as Neural Collaborative Filtering (NCF), FAISS-based similarity search, incorporation of user feedback mechanisms, and support for multiple languages.

References

- [1] A survey on reinforcement learning for recommender systems.
- [2] YANES, N., MOSTAFA, A.M., EZZ, M. and ALMUAYQIL, S.N. (2020) A machine learning-based recommender system for improving students learning experiences. *IEEE Access* 8: 201218–201235. doi:10.1109/ACCESS.2020.3036336.
- [3] Issues and solutions in deep learning-enabled recommendation systems within the e-commerce field .
- [4] GOODFELLOW, I., BENGIO, Y. and COURVILLE, A. (2016) *Deep Learning* (MIT Press). URL <https://www.deeplearningbook.org/>.
- [5] Deep learning-based recommendation system: Systematic review and classification.
- [6] RONKY FRANCIS, CONGHUA ZHOU, J.K.I.T.B.D. A systematic review of deep knowledge graph-based recommender systems, with focus on explainable embeddings. *mdpi* doi:2306-5729/7/7/94.
- [7] MU, R. (2018) A survey of recommender systems based on deep learning. *IEEE Access* 6: 69009–69022. doi:10.1109/ACCESS.2018.2880197.
- [8] QIAN ZHANG, JIE LU, Y.J. (2021) Artificial intelligence in recommender systems. *Springer* doi:<https://doi.org/10.1007/s40747-020-00212-w>.
- [9] ZHANG, S., YAO, L., SUN, A. and TAY, Y. (2019) Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys (CSUR)* 52(1): 5:1–5:38. doi:10.1145/3285029.
- [10] WEI LU, SHANSHAN CHEN, K.L.L.V.L. Show me the money: Dynamic recommendations for revenue maximization. *arxiv* .
- [11] BANIK, R. (2019) *Hands-On Recommendation Systems with Python* (Packt Publishing Ltd.).
- [12] PRASAD (2023), Retail transactions dataset, <https://www.kaggle.com/datasets/prasad22/retail-transactions-dataset>.
- [13] NOUSRESEARCH (2024), Nous-hermes-2-mixtral-8x7b-dpo, <https://huggingface.co/NousResearch/Nous-Hermes-2-Mixtral-8x7B-DPO>.
- [14] RICCI, F., ROKACH, L. and SHAPIRA, B. (2011) *Recommender Systems Handbook* (Springer).
- [15] JANNACH, D., ZANKER, M., FELFERNIG, A. and FRIEDRICH, G. (2010) *Recommender Systems: An Introduction* (Cambridge University Press).
- [16] SALTON, G. and MCGILL, M.J. (1986) *Introduction to Modern Information Retrieval* (New York, NY, USA: McGraw-Hill, Inc.).
- [17] VASWANI, A. et al. (2021) *Transformers for Natural Language Processing* (O’Reilly Media).
- [18] NAYSE, S.S. and DESHMUKH, M.P.R. (2023) Research paper recommendation system using transformer model. *Grenze International Journal of Engineering & Technology (GIJET)* 9(1).
- [19] HE, X., LIAO, L., ZHANG, H., NIE, L., HU, X. and CHUA, T.S. (2017) Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*: 173–182.
- [20] JAMES, G., WITTEN, D., HASTIE, T. and TIBSHIRANI, R. (2013) *An Introduction to Statistical Learning* (Springer).
- [21] OPENAI (2025), k-means clustering visualization, ChatGPT, AI-generated image. [Online]. Available: <https://chat.openai.com/>.
- [22] THORNDIKE, R.L. (1953) Who belongs in the family? *Psychometrika* 18(4): 267–276.
- [23] ROUSSEEUW, P.J. (1987) Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics* 20: 53–65.
- [24] GÉRON, A. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (O’Reilly Media).
- [25] MÜLLER, A.C. and GUIDO, S. (2016) *Introduction to Machine Learning with Python* (O’Reilly Media).